

AI 시대에 맞는 TDD 개발방법론

Agenda

- I. AI 시대에 TDD의 필요성
- II. AI 기반 TDD 실습 (Bowling KATA)
- III. Agentic Engineering은 Over-engineering일까?
- IV. 지속적인 발전을 위한 Agentic Engineering

교육 내용 및 학습 목표

- AI 시대에 TDD가 왜 다시 주목받고 있는지 살펴보고, AI 시대에 어울리는 TDD 방법론에 대해 알아본다.
- 끝으로 Agentic Engineering에 대한 방향성에 대해 살펴본다.

Chapter 01

AI 시대에 TDD의 필요성

SW 품질을 위한 Agentic Engineering

기존 TDD

TDD는 생산성보다 매우 높은 품질을 위한 방법론이다.

장점은 높은 SW 품질 확보이며,

단점으로는 테스트 코드 작성과 리팩토링에 드는 시간과 비용 때문에 느리고 비효율적이라는 인식이 존재했다.

AI 시대가 되면서 TDD가 더 편해졌다.

AI의 생산성의 도움을 받으면, 빠른 테스트코드 작성과 리팩토링이 가능하다.

이제 TDD는 더 이상 답답하고 부담스러운 방식이 아니다.

AI의 생산성이 결합되면서, 개발자는 높은 품질을 유지하면서도 실질적으로 활용 가능한 수준의 속도와 효율을 확보할 수 있게 되었다.

즉, AI 시대의 TDD는

“느리지만 좋은 방법”이 아니라,

“빠르면서도 품질을 보장하는 현실적인 전략”으로 재해석되고 있다.

1. 지시의 모호성 축소

- Text 기반으로 요구사항을 전달하면, 무엇을 만들어야 하는지 구체적이지 않을 수 있다.
- 반면 TDD는 Unit Test를 먼저 작성함으로써 AI에게 요구사항을 Code 기반으로 명확히 지시할 수 있다.
- 사람도 Text 지시에 대해 모호성을 겪지만, AI는 이를 더 크게 확대해석한다.
- 테스트는 “무엇을 만들어야 하는 지”를 명확하게 고정해준다.
- Text가 아닌 코드 기반 명세이기 때문에 해석 오차가 줄어든다.
- 즉, Text 지시가 아닌 Code로 지시하면, 잘못된 방향으로 구현될 확률을 낮춘다.

2. 안전장치 역할 (가장 중요)

- AI가 생성하는 코드는 빠르지만, 항상 신뢰할 수 있는 것은 아니다.
- TDD는 다음과 같은 구조를 만든다.
 - 먼저 테스트를 정의한다 (기대 결과 고정)
 - 그 다음 AI에게 구현을 맡긴다.
 - 여기서 Unit Test는 가드레일(안전장치) 역할을 한다.
 - 잘못된 구현은 Test Fail로 즉시 드러난다
 - 예상 범위를 벗어나는 코드가 만들어지기 어렵다.

3. 검토(리뷰)가 쉬워진다

- Test Code를 고민하면서 만들었다면, Green Code가 만들어지기 전 어떤 코드가 만들어질지 예상이 된다.
- 예측 범위 내에서 코드가 만들어지기에 AI가 만든 코드의 리뷰 속도가 굉장히 빠르다.

Chapter 02

AI 기반 TDD 실습 (Bowling KATA)

SW 품질을 위한 Agentic Engineering

AI 시대에 맞는 TDD

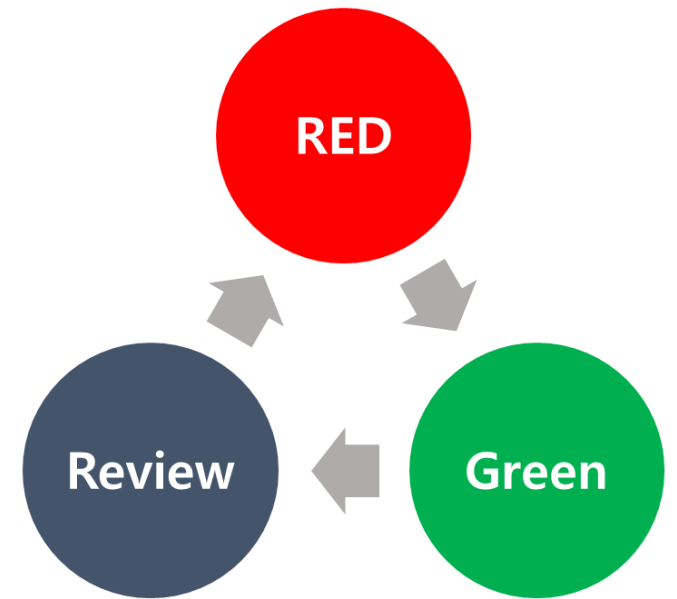
AI 시대에는 기존 방식에 더해, AI를 효과적으로 활용할 수 있는 Customize TDD가 필요하다.
이번 실습에서는 두 가지 방법의 AI + TDD를 단계적으로 경험해본다.

1. Original TDD 방식에 AI를 활용

- 기존 TDD 원칙을 유지한 상태에서 AI를 보조 도구로 활용한다.
- Bowling KATA를 통해 AI + TDD 사이클을 경험한다. (Superpower Skills 사용)

2. Agentic Engineering을 적용한 TDD 방식

- AE 원칙을 적용한 Red-Green-Review Cycle을 이해하고 경험한다.
- Bowling KATA를 Agentic Engineering 방식으로 재구성하여 실습한다.



Bowling kata 소개

Bowling 점수 산출 코드 작성 TDD

볼링 kata는 클린코드 저자로 유명한 Robert C. Martin 이 만든 TDD 연습용 kata이다.

실제 볼링 게임을 만드는 것이 아니라,

볼링 점수를 계산하는 로직과 설계를 TDD Cycle 로 연습해본다.

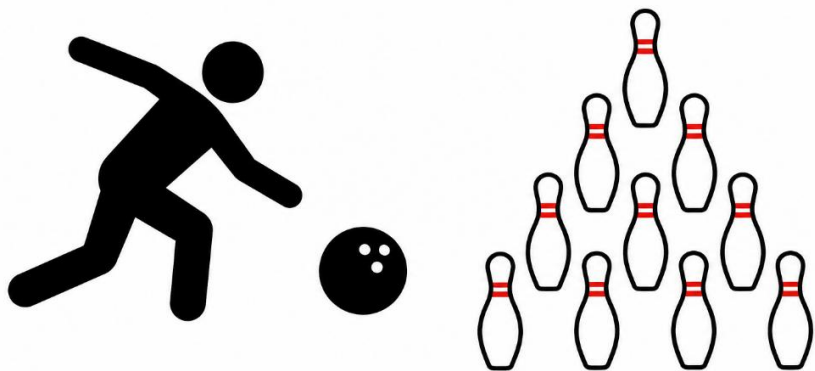
→ Coding Agent를 사용해서 TDD Cycle 을 돌려본다

Bowling 규칙

볼링 경기는 10 개의 볼링 핀을 넘어뜨리는 게임이다.

총 10 라운드가 진행되며, 각 라운드별로 최대 2번의 볼링공을 굴릴 기회가 주어진다.

다음 라운드에 볼링 핀은 다시 10개로 초기화된다.



각 라운드별로 **최대** 2번
굴릴 기회가 있다.

10 라운드는 예외로
최대 3번까지 굴릴 기회가 주어진다

1	2	3	4	5	6	7	8	9	10

볼링 점수판

기본 점수 규칙

한 라운드에서 얻을 수 있는 볼링의 점수는 공을 굴러 넘어뜨린 핀의 개수로 결정된다.
단, Spare 와 Strike 는 보너스 점수가 있다.

Round	
3	2

첫번째에는 3개, 두번째에는 2개를 쓰러뜨렸다.
→ 해당 라운드에서 얻은 점수 = $3 + 2$

Round	
2	/

첫번째에는 2개, 두번째에는 8개를 쓰러뜨려
한 라운드에 10개 모두 쓰러뜨렸다
→ 이를 Spare 라고 부른다.
→ 해당 라운드에서 얻은 점수 = $2 + 8 + \text{보너스}$

Round	
X	

첫번째 시도에 10개를 모두 쓰러뜨렸다.
→ 이를 Strike 라고 부른다.
→ 해당 라운드에서 얻은 점수 = $10 + \text{보너스}$

Spare 와 Strike 의 보너스 점수 규칙

한 라운드에 10개의 볼링 핀을 넘어뜨리면 Spare 또는 Strike 로 판정된다.

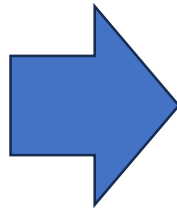
Spare 인 경우 **다음 1 번의 투구**에서 쓰러뜨린 핀 수를 보너스로 하고,

Strike 인 경우 **다음 2 번의 투구**에서 쓰러뜨린 핀 수를 보너스로 합산한다.

1Round		2Round	
2	/	3	5

1 Round 에서 얻는 점수
 $= 2 + 8 + 3$

1Round		2Round	
X		3	5



1 Round 에서 얻는 점수
 $= 10 + 3 + 5$

1Round		2Round		3Round	
X		X		X	

Q. 1,2,3 Round 에서 모두 Strike 인 경우
1 Round 에서 얻는 점수는?

마지막 10 Round 보너스

마지막 10 Round 는 Spare, Strike 인 경우 보너스 점수용 기회 포함 **3번 공을 굴릴 수 있다.**
단, 보너스 점수용 기회는 Strike이나 Spare이더라도 추가 기회는 없다.

10Round		
2	/	보너스

Spare 이므로 보너스 점수용 기회를 **1번** 준다.

10Round		
X	보너스	보너스

Strike 이므로 보너스 점수용 기회를 **2번** 준다.

Bowling 점수 예시

다음 볼링 점수판 예시를 보고 점수 규칙을 잘 이해했는지 확인해본다.

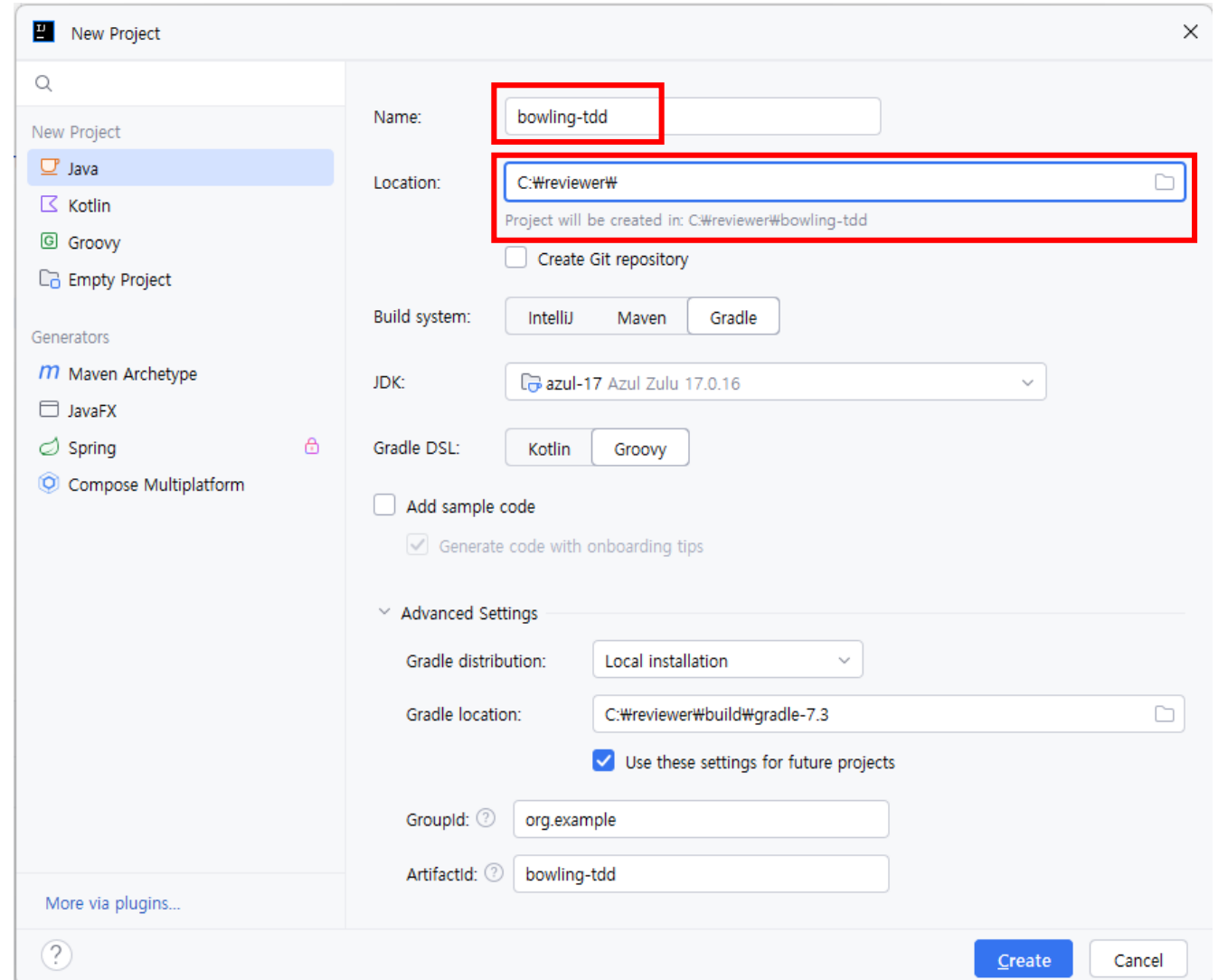
12	1	2	3	4	5	6	7	8	9	10	TTL
David	8 /	9 -	9 /	X	7 1	X	6 3	X	6 3	X 8 -	148
	19	28	48	66	74	93	102	121	130	148	
Smith	X	9 /	7 -	9 /	X	X	8 /	X	9 /	9 / ⑦	188
	20	37	44	64	92	112	132	152	171	188	
Green	4 -	5 -	8 -	X	1 8	1 -	6 /	8 -	9 -	7 1	89
	4	9	17	36	45	46	64	72	81	89	
Lazzari	9 /	5 4	3 /	7 /	X	6 -	6 -	⑧ 1	5 4		107
	15	24	41	61	77	83	89	98	107		
Hdcp	2/3	58	98	150	227	288	334	387	443	489	532

1 번째 쓰러뜨린 개수	2 번째 쓰러뜨린 개수
1 ~ n 라운드 누적점수	

Java 프로젝트 생성

Java 프로젝트 생성

Project Name : bowling-tdd



The image shows the 'New Project' dialog in IntelliJ IDEA. The 'Name' field is set to 'bowling-tdd' and the 'Location' field is set to 'C:\reviewer#'. The 'Build system' is set to 'IntelliJ', 'JDK' is 'azul-17 Azul Zulu 17.0.16', and 'Gradle DSL' is 'Kotlin'. The 'Advanced Settings' section shows 'Gradle distribution' as 'Local installation' and 'Gradle location' as 'C:\reviewer#build#gradle-7.3'. The 'Groupid' is 'org.example' and the 'Artifactid' is 'bowling-tdd'. The 'Create' button is highlighted in blue.

New Project

Java

Kotlin

Groovy

Empty Project

Generators

Maven Archetype

JavaFX

Spring

Compose Multiplatform

More via plugins...

Name: bowling-tdd

Location: C:\reviewer#

Project will be created in: C:\reviewer#\bowling-tdd

☐ Create Git repository

Build system: IntelliJ Maven Gradle

JDK: azul-17 Azul Zulu 17.0.16

Gradle DSL: Kotlin Groovy

☐ Add sample code

☒ Generate code with onboarding tips

Advanced Settings

Gradle distribution: Local installation

Gradle location: C:\reviewer#build#gradle-7.3

☒ Use these settings for future projects

Groupid: org.example

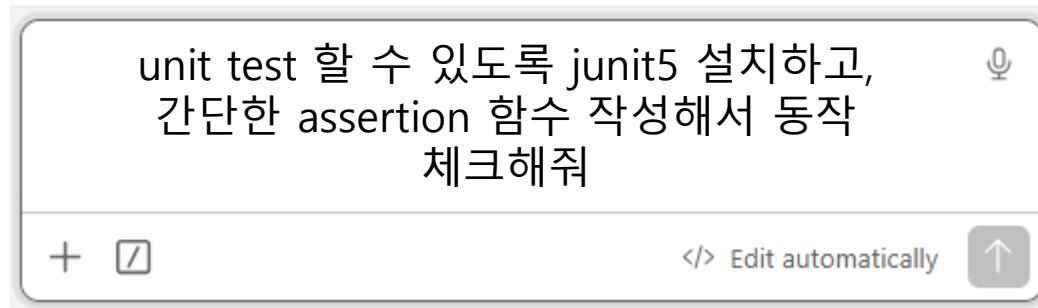
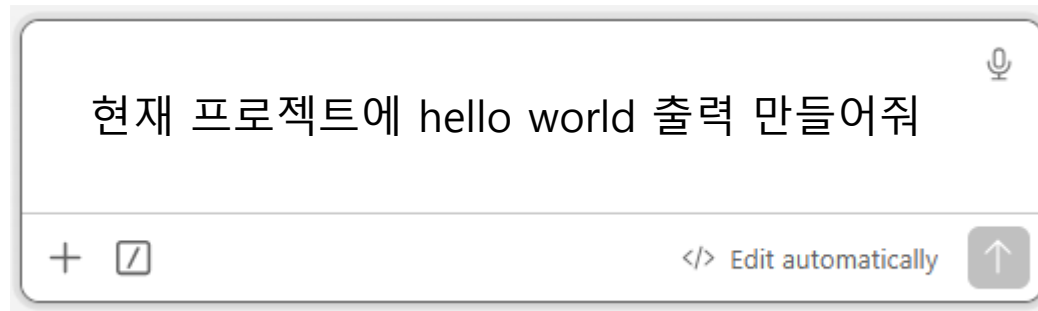
Artifactid: bowling-tdd

Create Cancel

프로젝트 세팅

agent 에게 기본 실습 세팅 요청하고, 정상적으로 동작되는지 체크까지 진행한다.

*** 동작 체크용 파일은 임시 파일이므로 지우도록 한다**

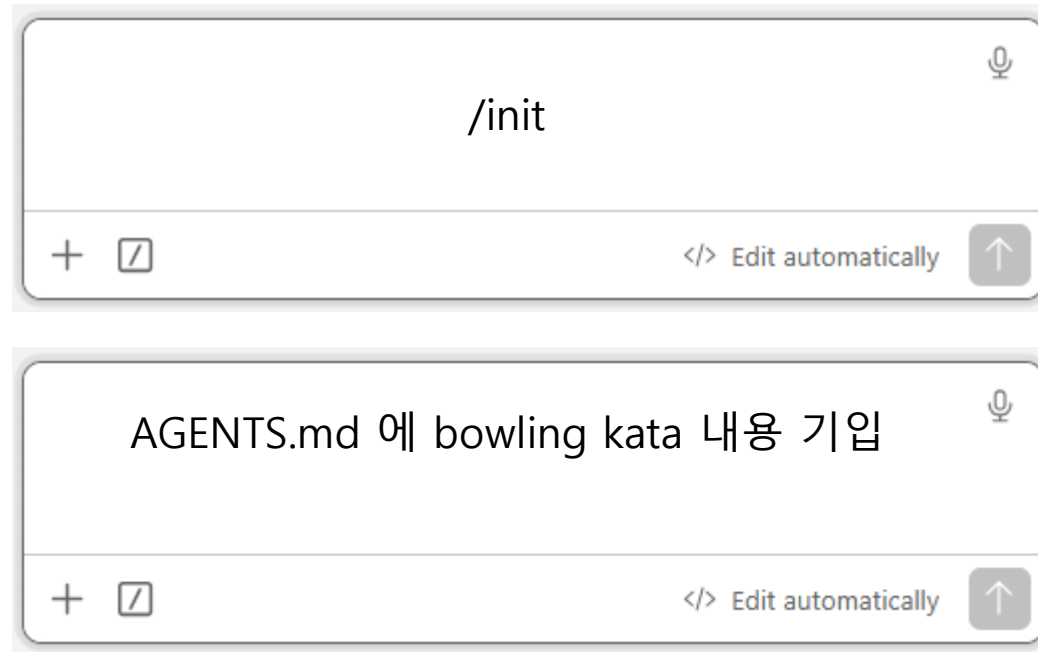


AGENTS.md 생성

/init skill을 사용하여 AGENTS.md 문서 생성 후

Bowling kata 내용 기입하기

기입할 내용: <https://gist.github.com/jeonghwan-seo/662e65f320fe0f0f3ce654ed464e89a8>



The image displays two sequential screenshots of a chat interface, likely from a development tool or AI assistant. The top screenshot shows a user inputting the command `/init`. The bottom screenshot shows the resulting output, which is the content of the `AGENTS.md` file, specifically mentioning the 'bowling kata' content being added. Both screenshots include a microphone icon in the top right corner and a bottom bar with a plus sign, a document icon, and a button labeled 'Edit automatically' with an upward arrow.

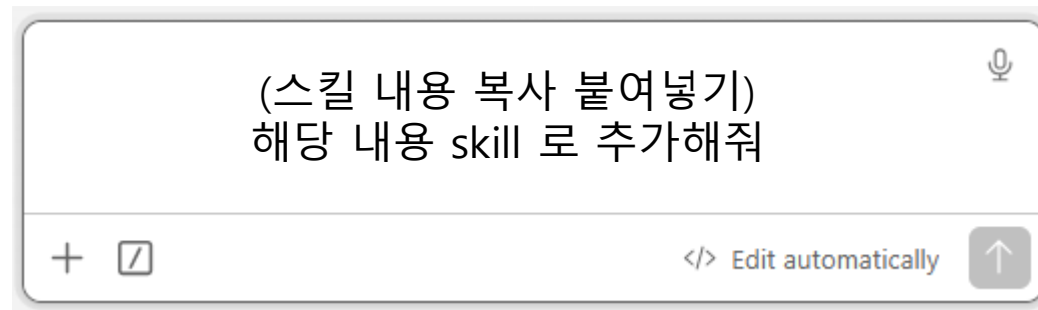
Top screenshot input: `/init`

Bottom screenshot output: AGENTS.md 에 bowling kata 내용 기입

Skill 세팅하기

이번 실습은 superpowers 의 TDD Skill 을 가져와서 사용해본다.

- superpowers TDD skill : <https://github.com/obra/superpowers/tree/main/skills/test-driven-development>
- 언어별 번역본 : <https://gist.github.com/jeonghwan-seo/fd1a2cd6a2d3cc00b8a3d5a68a21cc2f>

A screenshot of a skill creator interface. It features a large text input area with the placeholder text "(스킬 내용 복사 붙여넣기)" and "해당 내용 skill 로 추가해줘". To the right of the input area is a microphone icon. Below the input area is a horizontal bar containing a plus sign and a link icon on the left, the text "</> Edit automatically" in the center, and an upward arrow icon on the right.

skill creator 가 실행되어서 새로 만들어주거나,
기존내용을 그대로 사용하게 된다.

TDD Cycle 검토

기본적으로 Skill 에 명시된 대로 TDD 가 진행되는지 확인한다.

RED 단계에서 검토해야 할 부분 :

- ✓ 테스트는 구현에 대한 계획과 목표점이 된다.
- ✓ 테스트 작성을 제일 먼저 진행했는지
- ✓ 요구사항에 맞는 테스트 코드를 만들고 테스트 수행을 했는지

GREEN 단계에서 검토해야 할 부분:

- ✓ RED 단계의 테스트가 요구를 벗어난 불필요한 코드를 생성하진 않았는지 검토
- ✓ 테스트를 수행하고 PASS 되는지 확인

스킬 명시적 사용

스킬은 따로 언급하지 않아도 자연스럽게 발동될 수 있다.

하지만, 확실하게 사용하고 싶으면 명시적으로 스킬을 언급하도록 한다.

Bowling kata 구현해줘

+  </> Edit automatically 

```
> 볼링 카타 구현해줘
```

- 볼링 카타를 TDD 원칙대로 구현하겠습니다. `.claude/skills/test-driven-development` 가이드에 따라 RED → GREEN을 사이클별로 직접 보면서 진행합니다.

TDD 로 Bowling kata 구현해줘
test-driven-development 스킬 따라서 Bowling kata 구현해줘.
/test-driven-development Bowling kata 요구사항 파악하고
구현해줘

+  </> Edit automatically 

```
> tdd 로 볼링 카타 구현해줘
```

```
• Skill(test-driven-development)
  ↳ Successfully loaded skill
```



명시적인 사용

요구사항 파악 및 TDD 개발 진입

요구사항이 정리 되어있는 문서를 잘 읽고 이해했는지 파악한다.

- 먼저 현재 프로젝트 구조를 파악하겠습니다.

Searched for 3 patterns, read 2 files (ctrl+o to expand)

- 프로젝트 구조를 파악했습니다. Java 소스가 없는 깨끗한 스캐폴드입니다. CLAUDE.md의 볼링 요구사항을 바탕으로 TDD로 구현하겠습니다.

요구사항 정리:

- Game 클래스: `void roll(int pins), int score()`
- 스페어: 다음 1구 보너스
- 스트라이크: 다음 2구 보너스
- 10번 프레임: 스트라이크/스페어 시 추가 투구

TDD 사이클: 실패하는 테스트 → GREEN → 다음 테스트 순서로 진행합니다.

RED 1 – 거터 게임 (전부 0)

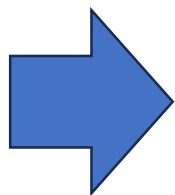
가장 단순한 기능에 대한 테스트를 추가하고, 테스트 실패를 확인한다.

- ✓ 스트라이크, 스페어 등의 여러 요구사항을 배제한 테스트부터 시작
- ✓ 코드 뼈대 형성 가능

RED 1 – 거터 게임 (전부 0)

먼저 테스트 파일을 작성합니다.

```
class GameTest {  
  
    private Game game;  
  
    @BeforeEach  
    void setUp() { game = new Game(); }  
  
    @Test  
    @DisplayName("거터 게임: 모든 투구가 0이면 총점은 0이다")  
    void gutterGame() {  
        rollMany( times: 20, pins: 0);  
        assertEquals(0, game.score());  
    }  
  
    private void rollMany(int times, int pins) {  
        for (int i = 0; i < times; i++) {  
            game.roll(pins);  
        }  
    }  
}
```



● RED 1 검증 – 실패 확인

● 다시 실패 확인 – **Game** 클래스가 없어서 컴파일 오류가 나야 합니다.

● Bash(cd "C:/reviewer/bowling-tdd" && ./gradlew.bat test --tests "org.example.bowling.GameTest.gutterGame" 2>&1)

└ Error: Exit code 1

> Task :compileJava NO-SOURCE

> Task :processResources NO-SOURCE

> Task :classes UP-TO-DATE

> Task :compileTestJava FAILED

C:\reviewer\bowling-tdd\src\test\java\org\example\bowling\GameTest.gutterGame
error: cannot find symbol

private Game game;

^

symbol: class Game

... +33 lines (ctrl+o to expand)

● **Game** 클래스 미존재로 정확히 실패합니다.

GREEN 1 – 최소 구현으로 통과

RED 1의 필요 구현 : “20번 볼링공을 굴린다”, “점수를 누적한다 ” 에 대한 구현을 한다.

GREEN 1 – 최소 구현으로 통과

- Write(src\main\java\org\example\bowling\Game.java)
 - └ Wrote 19 lines to src\main\java\org\example\bowling\Game.java

```
public class Game {  
  
    private int[] rolls = new int[21];  
    private int currentRoll = 0;  
  
    public void roll(int pins) {  
        rolls[currentRoll++] = pins;  
    }  
  
    public int score() {  
        int score = 0;  
        for (int i = 0; i < 20; i++) {  
            score += rolls[i];  
        }  
        return score;  
    }  
}
```

• GREEN 1 검증

- Bash(cd "C:/reviewer/bowling-tdd" && ./gradlew.bat test --tests "org.example.bowling.GameTest.gutterGame" 2>&1)
 - └ > Task :compileJava
 - > Task :processResources NO-SOURCE
 - > Task :classes
 - ... +20 lines (ctrl+o to expand)
 - └ (timeout 1m)
- 통과.

이전 RED 단계에서 작성한 테스트를 실행
→ PASS

RED 2 / GREEN 2 – 전부 1개의 볼링핀

점수 누적에 대한 테스트 추가

RED 2 – 전부 1핀 (모든 투구 1)

- Update(src\test\java\org\example\bowling\GameTest.java)
 - └ Added 7 lines

```
@Test
@DisplayName("모든 투구가 1이면 총점은 20이다")
void allOnes() {
    rollMany( times: 20, pins: 1);
    assertEquals( expected: 20, game.score());
}
```

- Bash(cd "C:/reviewer/bowling-tdd" && ./gradlew.bat test --tests "org.example.bowling.GameTest.allOnes" 2>&1)
 - └ > Task :compileJava
 - > Task :processResources NO-SOURCE
 - > Task :classes
 - ... +20 lines (ctrl+o to expand)
 - └ (timeout 1m)
- 기존 구현으로 자연스럽게 통과합니다 (20번 합산 로직이 이미 처리).

새로운 테스트, but 기존 구현으로 통과
→ 테스트가 이전 구현으로 우연히 통과인지 체크
→ 이전 구현이 너무 과도하지 않았는지 체크

RED 3 – 스페어 테스트 추가

스페어 점수에 대한 테스트 추가

RED 3 – 스페어

스페어 이후 다음 1구에 보너스가 없으면 틀린 점수가 나와야 합니다.

- Update(src\test\java\org\example\bowling\GameTest.java)
 - └ Added 10 lines

```
@Test
@DisplayName("스페어 후 다음 1구가 보너스로 추가된다")
void oneSpare() {
    game.roll( pins: 5);
    game.roll( pins: 5); // spare
    game.roll( pins: 3);
    rollMany( times: 17, pins: 0);
    assertEquals( expected: 16, game.score());
}
```

spare 의 보너스 규칙에 대한
테스트를 생성했는지 검토

```
● Bash(cd "C:/reviewer/bowling-tdd" && ./gradlew.bat test --tests
  "org.example.bowling.GameTest.oneSpare" 2>&1)
└ Error: Exit code 1
  > Task :compileJava UP-TO-DATE
  > Task :processResources NO-SOURCE
  > Task :classes UP-TO-DATE
  > Task :compileTestJava
  > Task :processTestResources NO-SOURCE
  > Task :testClasses

  > Task :test FAILED
```

- 예상대로 실패합니다 (보너스 미적용 → 13점, 기대값 16점).

스페어에 대한 기능 추가 전
→ FAIL 확인

GREEN 3 – 스페어 보너스 구현

Spare 점수에 대한 구현 추가

GREEN 3 – 스페어 보너스 구현

- Update(src\main\java\org\example\bowling\Game.java)
 - └ Added 13 lines, removed 2 lines

```
11
12     public int score() {
13         int score = 0;
14 -       for (int i = 0; i < 20; i++) {
15 -           score += rolls[i];
14 +       int rollIndex = 0;
15 +       for (int frame = 0; frame < 10; frame++) {
16 +           if (isSpare(rollIndex)) {
17 +               score += 10 + rolls[rollIndex + 2];
18 +               rollIndex += 2;
19 +           } else {
20 +               score += rolls[rollIndex] + rolls[rollIndex + 1];
21 +               rollIndex += 2;
22 +           }
23     }
24     return score;
25 }
26 +
27 + private boolean isSpare(int rollIndex) {
28 +     return rolls[rollIndex] + rolls[rollIndex + 1] == 10;
29 + }
30 }
31
```

● GREEN 3 검증

- Bash(cd "C:/reviewer/bowling-tdd" && ./gradlew.bat test 2>&1)
 - └ > Task :compileJava
 - > Task :processResources NO-SOURCE
 - > Task :classes
 - ... +20 lines (ctrl+o to expand)
 - └ (timeout 1m)
- 전체 통과.

RED 단계의 Spare 점수에 대한 테스트 실행
→ PASS 가 되어야 함

RED 4 – 스트라이크 테스트 추가

스트라이크 점수에 대한 테스트 추가

RED 4 – 스트라이크

- Update(src\test\java\org\example\bowling\GameTest.java)
 - └ Added 10 lines

```
@Test
@DisplayName("스트라이크 후 다음 2구가 보너스로 추가된다")
void oneStrike() {
    game.roll(pins: 10); // strike
    game.roll(pins: 3);
    game.roll(pins: 4);
    rollMany(times: 16, pins: 0);
    assertEquals(expected: 24, game.score());
}
```

스트라이크 보너스 점수에 대한
테스트를 생성했는지 검토

```
● Bash(cd "C:/reviewer/bowling-tdd" && ./gradlew.bat test --tests
  "org.example.bowling.GameTest.oneStrike" 2>&1)
└ Error: Exit code 1
  > Task :compileJava UP-TO-DATE
  > Task :processResources NO-SOURCE
  > Task :classes UP-TO-DATE
  > Task :compileTestJava
  > Task :processTestResources NO-SOURCE
  > Task :testClasses

  > Task :test FAILED

... +24 lines (ctrl+o to expand)
```

- 예상대로 실패합니다.

스트라이크에 대한 기능 추가 전
→ FAIL 확인

GREEN 4 – 스트라이크 보너스 구현

스트라이크 기능에 대한 테스트가 통과할 수 있도록 구현

GREEN 4 – 스트라이크 보너스 구현

- Update(src\main\java\org\example\bowling\Game.java)
└ Added 8 lines, removed 1 line

```
public int score() {  
    int score = 0;  
    int rollIndex = 0;  
    for (int frame = 0; frame < 10; frame++) {  
        if (isStrike(rollIndex)) {  
            score += 10 + rolls[rollIndex + 1] + rolls[rollIndex + 2];  
            rollIndex += 1;  
        } else if (isSpare(rollIndex)) {  
            score += 10 + rolls[rollIndex + 2];  
            rollIndex += 2;  
        } else {  
            score += rolls[rollIndex] + rolls[rollIndex + 1];  
            rollIndex += 2;  
        }  
    }  
    return score;  
}  
  
private boolean isStrike(int rollIndex) {  
    return rolls[rollIndex] == 10;  
}
```

● GREEN 4 검증

- Bash(cd "C:/reviewer/bowling-tdd" && ./grad
└ > Task :compileJava
 > Task :processResources NO-SOURCE
 > Task :classes
 ... +20 lines (ctrl+o to expand)
└ (timeout 1m)

● 전체 통과.

RED/GREEN 5 – 퍼펙트 게임 테스트(모두 스트라이크)

마지막 10라운드에서의 규칙도 적용되도록 테스트와 구현을 한다.

RED 5 – 퍼펙트 게임 (모두 스트라이크)

- Update(src\test\java\org\example\bowling\GameTest.java)
 - └ Added 7 lines
 - 23 }

```
@Test
@DisplayName("퍼펙트 게임: 12번 연속 스트라이크이면 총점은 300이다")
void perfectGame() {
    rollMany(times: 12, pins: 10);
    assertEquals(expected: 300, game.score());
}
```

```
● Bash(cd "C:/reviewer/bowling-tdd" && ./gradlew.bat test --tests
  "org.example.bowling.GameTest.perfectGame" 2>&1)
└ > Task :compileJava UP-TO-DATE
  > Task :processResources NO-SOURCE
  > Task :classes UP-TO-DATE
  ... +20 lines (ctrl+o to expand)
└ (timeout 1m)
```

- 퍼펙트 게임도 기존 구현으로 이미 처리됩니다 (21-슬롯 배열이 10번 프레임 보너스 투구를 수용).

전체 테스트가 통과했습니다. 최종 상태를 확인합니다.

새로운 테스트, but 기존 구현으로 통과
→ 테스트가 이전 구현으로 우연히 통과인지 체크
→ 이전 구현이 너무 과도하지 않았는지 체크

구현 요약

마지막에 Agent 가 구현 요약에 대한 리포트를 준다.

- TDD 사이클 완료 . 체크리스트 확인 :

- 모든 새 메서드/클래스에 테스트가 있다
- 각 테스트가 구현 전에 실패하는 것을 직접 보았다
- 각 테스트가 예상한 이유로 실패했다 (기능 부재)
- 각 테스트를 통과시킬 최소 코드를 작성했다
- 모든 테스트가 통과한다 (`./gradlew.bat test`)

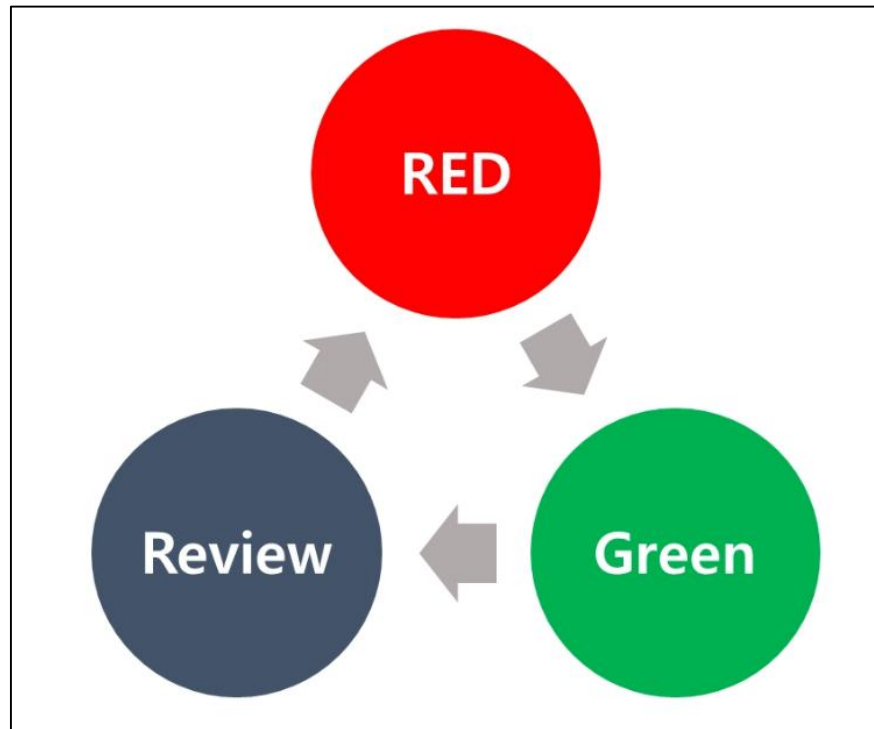
구현 완료 요약:

사이클	테스트	결과
RED→GREEN 1	거터 게임 (20구 0핀 → 0점)	최소 구현 (<code>rolls</code> 배열 할산)
RED→GREEN 2	전부 1핀 (20구 → 20점)	기존 구현으로 자동 통과
RED→GREEN 3	스페어 (5+5+3+... → 16점)	스페어 보너스 로직 추가
RED→GREEN 4	스트라이크 (10+3+4+... → 24점)	스트라이크 보너스 로직 추가
RED→GREEN 5	퍼펙트 게임 (12×10 → 300점)	기존 구현으로 자동 통과

`Game` 클래스는 10개 프레임을 순회하며 스트라이크→스페어→일반 순서로 판별하고, 21-슬롯 배열이 10번 프레임 보너스 투구를 자연스럽게 수용합니다.

Agentic Engineering TDD 리뷰

human-in-the-loop 를 강조하여 Agent 를 사용한 TDD 를 진행해본다.



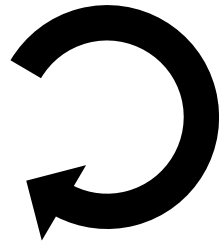
1. RED
 - 목표 정확히 설정
 - Plan.md 생성 요청 후 검토
2. Green
 - RED 에서 생성된 goal 을 달성하도록 구현
 - 테스트를 실제로 통과하는지 체크
3. Review
 - Green 의 코드를 리뷰
 - Plan 외의 사항이 구현되었는지
 - 리팩토링이 필요하다면 요청하기

프로젝트 수행시 커밋할 부분

agentic engineering 수행을 체크하기 위한 용도로,
각 단계별 필요한 커밋들을 생성한다.



RED 단계에서 생성된
Plan.md Commit



GREEN 단계의 코드를 검토 후 Commit



프로젝트 제출방법 - 프로젝트

GitHub 에 Remote Repository를 생성

Repository 이름

- repo 이름 : **bowling-사번**
- Description : 소속 & 이름
- 접근 권한 : Public

Create a new repository

A repository contains all project files, including the revision history.

Owner

Repository name *

 reviewer-01 ▾

 /

bowling-사번

Great repository names are short and memorable. Need inspiration? How about [redesigned-journey?](#)

Description (optional)

소속 & 이름

☒  Public

Any logged in user can see this repository. You choose who can commit.

☐  Internal

Samsung SDS [enterprise members](#) can see this repository. You choose who can commit.

☐  Private

You choose who can see and commit to this repository.

Skip this step if you're importing an existing repository.

☐ Initialize this repository with a README

This will let you immediately clone the repository to your computer.

Add .gitignore: None ▾

Create repository

Skill Update하기

human-in-the-loop 를 강조하여 TDD 를 진행하기 위해,
필요한 내용들을 적은 후 기존 tdd 스킬을 업데이트한다.

```
> @skill-update-plan.md 를 파악해서 local project 의 /test-driven-development skill 을 업데이트  
해줘  
L Read skill-update-plan.md (32 lines)
```

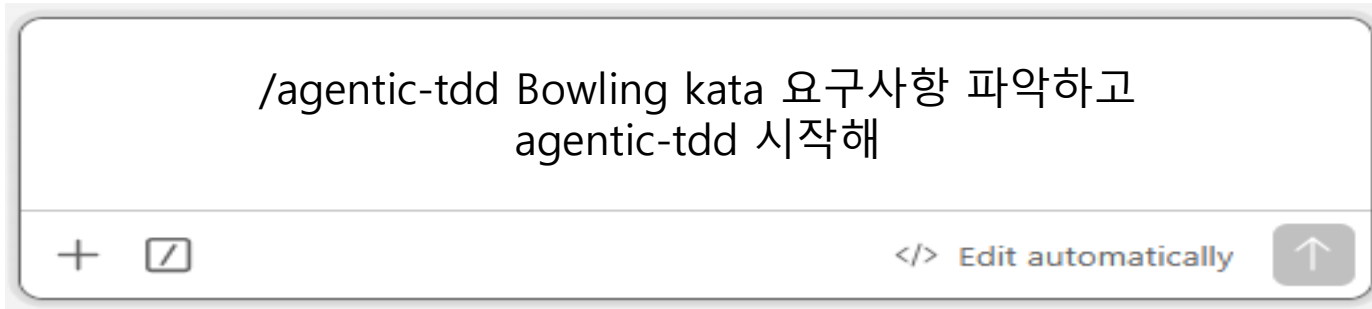
agentic engineering 을 TDD 에서 실천할 수 있도록
문서를 작성해놓는다.

새롭게 작성된 Skill 가동

update 된 skill 을 사용해서 Agentic Engineering 에 맞는 TDD 를 진행한다.

- 한 사이클을 수행하면서 Agentic Engineering 을 잘 지키는지 Skill 검토가 필요하다.
- Agentic Engineering 을 잘 수행하지 않으면 Skill을 수정한다.

```
/agentic-tdd Bowling kata 요구사항 파악하고  
agentic-tdd 시작해
```



RED 1 – Plan.md

Agent 가 작성한 Plan.md 를 검토한다.

● RED – Plan.md (첫 번째 테스트)

기능: 거터 게임(Gutter Game) – 20번 모두 0핀, 총점 = 0

근거:

볼링 Game 클래스의 roll(int pins)와 score() 메서드가 아직 존재하지 않습니다. 가장 단순한 케이스인 "한 번도 핀을 쓰러뜨리지 못한 게임"부터 시작해 인터페이스를 확립합니다. 이 테스트 하나로 Game 클래스 뼈대, roll() 호출, score() 반환값의 기본 구조를 강제합니다.

테스트 설계 (코드 작성 전 의도 기술):

```
Game game = new Game();
20번 roll(0) 호출
assertEquals(0, game.score())
```

패키지: org.example.bowling

⊖ 이 Plan을 검토하고 승인해 주세요. 승인 후 테스트 코드로 변환하겠습니다.

Plan.md — 첫 번째 TDD 사이클

기능

거터 게임 (Gutter Game) — 20번 모두 0핀, 총점 = 0

근거

볼링 Game 클래스의 roll(int pins) 와 score() 메서드가 아직 존재하지 않습니다. 가장 단순한 케이스인 "한 번도 핀을 쓰러뜨리지 못한 게임"부터 시작해 인터페이스를 확립합니다. 이 테스트 하나로 Game 클래스 뼈대, roll() 호출, score() 반환값의 기본 구조를 강제합니다.

테스트 설계 (코드 작성 전 의도)

```
Game game = new Game();
20번 roll(0) 호출
assertEquals(0, game.score())
```

패키지

```
org.example.bowling
```

RED 1 – Test Code 변환 후 검토

Plan.md 가 마음에 들었다면 다음 step 을 진행시킨다.

Plan.md → Test Code (plan) 으로 변환작업을 수행

사람은 Plan 에 맞게 Test Code 를 만들었는지 확인한다.



GREEN 1 – 테스트를 통과시킬 최소 구현

Test Code 를 기반으로 빠르게 코드를 생성 후, Test Code 를 돌려본다.

```
public class Game {  
  
    private int score = 0;  
  
    public void roll(int pins) {  
        score += pins;  
    }  
  
    public int score() {  
        return score;  
    }  
}
```

```
● Bash(./gradlew.bat test --tests "org.example.bowling.GameTest  
  --rerun-tasks 2>&1)  
L > Task :compileJava  
> Task :processResources NO-SOURCE  
> Task :classes  
... +16 lines (ctrl+o to expand)  
● GREEN 확인 완료 - BUILD SUCCESSFUL
```

REVIEW 1 – 생성된 코드 리뷰

코드를 검토한다.

Plan(Test Code) 에 맞도록 구현되었는가

- 지나치게 일반화 되거나, 불필요한 부분을 추가하지 않았는지
- 리팩토링이 필요한지

❌ CODE REVIEW - 코드를 검토해 주세요.

검토 후 다음 사이클(두 번째 테스트)로 넘어가겠습니다.

```
public class Game {  
  
    private int score = 0;  
  
    public void roll(int pins) {  
        score += pins;  
    }  
  
    public int score() {  
        return score;  
    }  
}
```



RED 2 – Plan.md

다음 사이클에서도 Plan.md 를 받는다.

```
> 다음 사이클로 넘어가

• RED - Plan.md (두 번째 사이클)

기능: 모든 투구가 1핀인 게임 - 20번 모두 1핀, 총점 = 20

근거:
거터 게임은 score()가 항상 0을 반환해도 통과합니다. 이 테스트
누적하고 있음을 강제합니다. 스페어·스트라이크 없이 순수 합산
"정상 케이스"입니다.

• Write(Plan.md)
  L Added 7 lines, removed 7 lines
  1 -# Plan.md - 첫 번째 TDD 사이클
  1 +# Plan.md - 두 번째 TDD 사이클
  2
  3 ## 기능
  4 -거터 게임 (Gutter Game) - 20번 모두 0핀, 총점 = 0
  4 +모든 투구가 1핀인 게임 - 20번 모두 1핀, 총점 = 20
  5
  6 ## 근거
  7 -볼링 'Game' 클래스의 'roll(int pins)'와 'score()' 메서드
```

Plan.md — 두 번째 TDD 사이클

기능

모든 투구가 1핀인 게임 — 20번 모두 1핀, 총점 = 20

근거

거터 게임은 score()가 항상 0을 반환해도 통과합니다. 이 테스트는 roll()이 실제로 핀 수를 누적하고 있음을 강제합니다. 스페어·스트라이크 없이 순수 합산만 검증하는 가장 단순한 "정상 케이스"입니다.

테스트 설계 (코드 작성 전 의도)

```
Game game = new Game();
20번 roll(1) 호출
assertEquals(20, game.score())
```

패키지

```
org.example.bowling
```

RED 2 – Test Code 변환 후 검토

마찬가지로 변환된 Test Code 가 Plan.md 의 목표에 맞도록 작성되었는지 확인한다.



The screenshot displays an IDE with two panels. The left panel shows a Markdown file named 'Plan.md' with the following content:

Plan.md — 두 번째 TDD 사이클

기능

모든 투구가 1핀인 게임 — 20번 모두 1핀, 총점 = 20

근거

거터 게임은 score()가 항상 0을 반환해도 통과합니다. 이 테스트는 roll()이 실제로 핀 수를 누적하고 있음을 강제합니다. 스페어·스트라이크 없이 순수 합산만 검증하는 가장 단순한 "정상 케이스"입니다.

테스트 설계 (코드 작성 전 의도)

```
Game game = new Game();
20번 roll(1) 호출
assertEquals(20, game.score())
```

패키지

```
org.example.bowling
```

The right panel shows a Java file named 'GameTest.java' with the following code:

```
1 package org.example.bowling;
2
3 import org.junit.jupiter.api.DisplayName;
4 import org.junit.jupiter.api.Test;
5
6 import static org.junit.jupiter.api.Assertions.assertEquals;
7
8 class GameTest {
9
10     @Test
11     @DisplayName("거터 게임은 총점이 0이다")
12     void gutterGameScoresZero() {
13         Game game = new Game();
14         for (int i = 0; i < 20; i++) {
15             game.roll(pins: 0);
16         }
17         assertEquals(expected: 0, game.score());
18     }
19
20     @Test
21     @DisplayName("모든 투구가 1핀이면 총점이 20이다")
22     void allOnesScoresTwenty() {
23         Game game = new Game();
24         for (int i = 0; i < 20; i++) {
25             game.roll(pins: 1);
26         }
27         assertEquals(expected: 20, game.score());
28     }
29 }
30
```

RED→REVIEW 2 – 테스트 검토

작성된 테스트코드가 GREEN 단계를 거치지 않고 통과되었다.
테스트가 유효한 테스트인지 확인후 넘어간다.

- ⚠ 테스트가 즉시 통과했습니다.

현재 구현이 핀을 단순 합산하므로 이 케이스는 이미 올바르게 동작합니다. TDD 규칙상 "즉시 통과 = 이미 존재하는 동작 테스트" 입니다.

이 테스트를 회귀 방지용으로 유지하되, 다음 사이클로 바로 넘어가는 것을 권장합니다. 현재 구현이 검증하지 못하는 첫 케이스는 스페어 보너스입니다.

다음 `Plan.md`를 작성할까요?

RED 3 – Plan.md 를 테스트로

마찬가지로 Plan.md 를 검토 후 Test 생성을 한다.

Plan.md — 세 번째 TDD 사이클

기능

스페어 보너스 — 스페어 이후 다음 투구 1회가 보너스로 추가된다

근거

현재 구현은 핀 수를 단순 합산합니다. 스페어(한 프레임에서 2회 투구로 10핀)가 발생하면 다음 투구 1회의 핀 수가 해당 프레임 점수에 보너스로 더해져야 합니다. 이 테스트는 단순 합산으로는 통과할 수 없으며, 프레임 구조 인식을 강제합니다.

테스트 설계 (코드 작성 전 의도)

1프레임: roll(5), roll(5) → 스페어
2프레임: roll(3), roll(0) → 보너스 3점
나머지 8프레임: roll(0) * 16

기대 총점 = 10(스페어) + 3(보너스) + 3(2프레임) = 16

패키지

org.example.bowling

```
@Test
@DisplayName("스페어 이후 다음 투구가 보너스로 추가된다")
void spareFollowedByThreeScoresSixteen() {
    Game game = new Game();
    game.roll(pins: 5);
    game.roll(pins: 5); // 스페어
    game.roll(pins: 3);
    game.roll(pins: 0);
    for (int i = 0; i < 16; i++) {
        game.roll(pins: 0);
    }
    assertEquals(expected: 16, game.score());
}
```

GREEN 3 – 테스트 통과 확인

Test Code 가 잘 통과되는지 확인한다.

```
● Bash(./gradlew.bat test --rerun-tasks 2>&1)
  L > Task :compileJava
    > Task :processResources NO-SOURCE
    > Task :classes
    ... +17 lines (ctrl+o to expand)

● ● GREEN 확인 완료 - 전체 테스트 BUILD SUCCESSFUL
```

REVIEW 3

생성된 코드에 대해 리뷰한다.

```
public class Game {  
  
    private int[] rolls = new int[21];  
    private int currentRoll = 0;  
  
    public void roll(int pins) {  
        rolls[currentRoll++] = pins;  
    }  
  
    public int score() {  
        int score = 0;  
        int rollIndex = 0;  
        for (int frame = 0; frame < 10; frame++) {  
            if (rolls[rollIndex] + rolls[rollIndex + 1] == 10) {  
                score += 10 + rolls[rollIndex + 2];  
                rollIndex += 2;  
            } else {  
                score += rolls[rollIndex] + rolls[rollIndex + 1];  
                rollIndex += 2;  
            }  
        }  
        return score;  
    }  
}
```

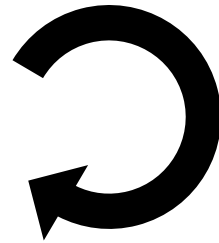

나머지 Cycle

나머지 Cycle 을 과제로 수행하도록 한다.

과제 수행 시 human-in-the-loop 가 잘 수행되었는지 체크용으로 Commit 을 만든다.



RED 단계에서 생성된
Plan.md Commit



GREEN 단계의 코드를 검토 후 Commit



Chapter 04

지속적인 발전을 위한 Agentic Engineering

SW 품질을 위한 Agentic Engineering

Agentic Engineering의 단점

- 똑딱 만드는 하네스 엔지니어링에 비해, 개발 속도가 매우 느리다.
- Self 검토 및 AI와 토론, 팀 동료와 토론 및 공유 시간이 많이 들기 때문이다.
- 클린 코드 일수록 이 시간을 줄일 수 있지만, 여전히 리뷰 시간은 길다.

하지만 SW 품질을 위해 반드시 필요한 투자 시간이다.

SW품질 확보 외, Agentic Engineering의 장점

실력이 꾸준히 향상된다. AI로 인해 실력이 정체되는 것을 방지할 수 있다.

- 연차가 쌓일수록 SW 검토 실력이 지속적으로 늘어야 한다. 이건 중요한 문제이다.

반면, 하네스 엔지니어링에 과도하게 의존할 경우 코드 구현 역량이 점차 약화될 수 있다.

구현 경험이 줄어들수록 코드의 동작을 깊이 이해하는 능력과 검토, 판단 역량 역시 함께 약화될 가능성이 높다.

피할 수 없는 개발 인력의 축소, 어떻게 대응할까?

AI와 협업으로, 적은 인원으로도 SW 개발이 가능하다.

하네스 엔지니어링이 발전할수록 적은 실력으로도, OS급 SW개발도 가능해진다.

이런 발전 속도에서 SW 개발자의 경쟁력은 어디서 나올까?

- AI 없이도 개발할 수 있는 기본 SW 훈련 수준
 - SW 훈련이 잘 되어있을 수록 더 심도 있는 수준의 AI 토론이 가능하다.
- AI에게 사고를 의존하지 않고, AI 작업물에 대해 비판적 사고를 갖는다.
 - AI에게 냉철하게 질문한다.
- 알고리즘, 자료구조 능력 UP / 문제 분해 능력
 - AI가 만들 구현방식이 미리 예상이 되며, 이에 맞추어 Phase 분리를 잘 한다.
- AI가 만든 작업물을 검토하는 능력
 - 클린코드에 대한 고민을 많이 하여, 코드를 빠르게 이해하고 클린코드 방향성을 빠르게 캐치하는 능력

Agentic Engineering은 개발 역량을 지속적으로 올릴 수 있는 엔지니어링이다.

감사합니다.